

Fiche Individuelle de Contribution au Projet

Projet : Conception d'une architecture distribuée avec routage en oignon "Axolotl"

Étudiante : Soumaya RABAH

Formation : BUT 2 Réseaux et Télécommunications

Période : 13 novembre 2025 - 30 décembre 2025

Binôme : Nolan HOARAU

FICHE INDIVIDUELLE DE CONTRIBUTION AU PROJET	1
INTRODUCTION : MON ROLE DANS LE PROJET	2
CORPS : DESCRIPTION DETAILLEE DE MES CONTRIBUTIONS	2
1. ORGANISATION DU PROJET ET GESTION DOCUMENTAIRE.....	2
2. BASE DE DONNEES MARIADB	3
3. INTERFACES GRAPHIQUES PYQT6.....	3
<i>Interface du Master</i>	4
<i>Interface du Client</i>	4
4. INTEGRATION ET TESTS COLLABORATIFS.....	5
5. DOCUMENTATION ET LIVRABLES	6
ANALYSE DE LA COMPETENCE AU REGARD DES AC/CE	6
AC23.01 AUTOMATISER L'ADMINISTRATION SYSTEME AVEC DES SCRIPTS.....	6
AC23.02 DEVELOPPER UNE APPLICATION A PARTIR D'UN CAHIER DES CHARGES DONNE	6
AC23.03 UTILISER UN PROTOCOLE RESEAU POUR PROGRAMMER UNE APPLICATION CLIENT/SERVEUR	7
AC23.04 INSTALLER, ADMINISTRER UN SYSTEME DE GESTION DE DONNEES.....	7
AC23.05 ACCEDER A UN ENSEMBLE DE DONNEES DEPUIS UNE APPLICATION	7
CE3.01 EN ETANT A L'ECOUTE DES BESOINS DU CLIENT	7
CE3.02 EN DOCUMENTANT LE TRAVAIL REALISE	8
CE3.03 EN UTILISANT LES OUTILS NUMERIQUES A BON ESCIENT	8
CE3.04 EN CHOISSANT LES OUTILS DE DEVELOPPEMENT ADAPTES	8
CE3.05 EN INTEGRANT LES PROBLEMATIQUES DE SECURITE	8
CONCLUSION SUR MON APPRENTISSAGE	8

Introduction : Mon rôle dans le projet

Dans ce projet de développement d'un système de communication anonyme avec routage en oignon, j'ai occupé le rôle de développeuse spécialisée en interfaces utilisateur et en gestion de l'organisation du projet.

Tandis que Nolan assurait la coordination technique en vérifiant la cohérence du code et l'intégration des différentes parties, je me suis concentrée sur l'expérience utilisateur en concevant et développant l'intégralité des interfaces graphiques avec PyQt6. J'étais également responsable de l'organisation pratique du projet : gestion du dépôt Git, structuration des fichiers, planification documentée et rédaction des livrables.

En complément du travail de Nolan sur la logique réseau et cryptographique, j'ai mis en place la base de données MariaDB et assuré l'intégration des interfaces avec le backend. Cette répartition complémentaire nous a permis de travailler efficacement en parallèle tout en restant synchronisés via des stand-ups quotidiens sur Discord.

Corps : Description détaillée de mes contributions

1. Organisation du projet et gestion documentaire

Pendant que Nolan posait les fondations techniques du projet avec les sockets et threads, je me suis occupée de structurer l'environnement de travail pour faciliter notre collaboration. J'ai créé l'architecture complète du dépôt Git avec une organisation claire : dossiers `source/` pour le code Python, `database/` pour les scripts SQL, `docs/` pour la documentation. J'ai établi des conventions de commit claires (`feat:`, `fix:`, `docs:`, `refactor:`, `test:`) que nous avons suivies tous les deux tout au long du projet, permettant un historique Git lisible et professionnel.

J'ai mis en place un système de branches (`main` pour la production, `dev` pour l'intégration, `feature/*` pour les développements) qui nous a permis de travailler sans conflit. Sur les 85 commits du projet, j'en ai réalisé 42, principalement concentrés sur les interfaces, la base de données et la documentation.

Pour maintenir notre synchronisation, j'ai organisé des stand-ups quotidiens sur Discord à 18h30 (10-15 minutes). Ces sessions structurées nous permettaient de partager nos avancements, identifier rapidement les blocages et planifier des sessions de pair programming quand nécessaire. Par exemple, lorsque j'avais des difficultés avec les signaux Qt pour l'intégration réseau, Nolan et moi nous mettions en partage d'écran pour résoudre le problème ensemble.

J'ai rédigé le planning préparatoire rendu le 20 novembre, avec l'analyse complète du cahier des charges, un diagramme de Gantt détaillé sur 8 phases (48 jours), la répartition claire des tâches entre Nolan et moi, et l'identification des risques potentiels. Ce document nous a servi de feuille de route tout au long du projet.

Enfin, j'ai contribué massivement à la documentation finale : le rapport technique de 1800+ lignes (sections sur l'architecture des interfaces, la base de données et le bilan de gestion de projet), le README avec instructions d'installation, et le guide utilisateur avec captures d'écran. Cette documentation complète permet à quiconque de comprendre, installer et utiliser notre système.

2. Base de données MariaDB

Pendant que Nolan développait la logique de chiffrement RSA et le routage en oignon, j'ai conçu et implémenté toute la couche de persistance des données. J'ai analysé les besoins du système pour identifier les entités à stocker : routeurs avec leurs clés cryptographiques, utilisateurs avec leur statut de connexion, messages pour la journalisation, et routes pour l'historique des chemins.

J'ai créé un schéma relationnel complet avec quatre tables principales :

- `routers` : stocke l'ID, IP, port, et les trois composantes des clés RSA (e, n, d), avec horodatage automatique
- `users` : contient username, IP, port, clés publiques (e, n), et un champ booléen `is_online` pour gérer le statut en temps réel
- `messages` : permet la journalisation optionnelle des échanges avec expéditeur, destinataire et chemin
- `routes` : conserve l'historique des routes créées pour analyse ultérieure

Mes choix de conception étaient justifiés techniquement : `VARCHAR(45)` pour supporter IPv4 et IPv6, `BIGINT` pour les clés RSA potentiellement grandes, `TIMESTAMP` avec auto-update pour le suivi automatique sans intervention manuelle, et `UNIQUE KEY` sur (ip, port) pour éviter les doublons de routeurs.

J'ai rédigé le fichier `schema.sql` (80 lignes) avec toutes les contraintes, index d'optimisation et commentaires explicatifs, le fichier `config.py` centralisant les paramètres de connexion, et `test_db.py` pour valider la connexion et l'accès aux tables avant l'intégration.

Dans le code Python, j'ai implémenté les fonctions d'accès à la base de données dans `master.py` : `get_db()` pour établir la connexion avec gestion d'erreurs, `initialize_database()` pour créer automatiquement les tables au premier lancement, et `clear_database_tables()` pour nettoyer les données au redémarrage. J'ai également contribué aux requêtes `SELECT` simples pour récupérer les listes de routeurs et utilisateurs, pendant que Nolan gérait les opérations d'insertion lors des enregistrements.

3. Interfaces graphiques PyQt6

C'est ma contribution principale et la plus conséquente du projet. Alors que Nolan se concentrait sur le cœur technique (chiffrement, sockets, routage), je me suis spécialisée dans tout l'aspect visuel et d'expérience utilisateur.

PyQt6 était une technologie entièrement nouvelle pour moi. J'ai consacré environ 10 heures à un apprentissage progressif : documentation officielle, tutoriels, et expérimentation. J'ai commencé par les widgets de base (`QLabel`, `QPushButton`, `QLineEdit`), puis les layouts pour organiser l'interface, ensuite les widgets complexes (`QTableWidget`, `QTextEdit`), et enfin les concepts avancés comme les signaux/slots et la programmation thread-safe.

Interface du Master

J'ai développé deux fenêtres complètes pour le Master. La première, `MasterLoginWindow`, est une fenêtre de configuration apparaissant avant le démarrage du serveur. Elle permet à l'administrateur de personnaliser l'IP et le port du Master avec des champs pré-remplis (127.0.0.1:6000), une validation rigoureuse du format IP via `socket.inet_aton()`, et une vérification de la plage de port (1-65535). Les messages d'erreur utilisent `QMessageBox` pour guider l'utilisateur de manière claire et professionnelle.

La seconde fenêtre, `MasterWindow`, est l'interface principale de monitoring du système. Elle comporte un système d'onglets moderne : l'onglet "Logs" affiche tous les événements du serveur en temps réel dans un `QTextEdit` avec police monospace et auto-scroll automatique vers le bas pour suivre l'activité. L'onglet "Connexions" présente une vue d'ensemble du réseau avec les informations du Master en haut (IP, port, statut avec un point vert), puis deux tableaux disposés côte à côte : le tableau des routeurs affiche ID, IP/Port et État, tandis que le tableau des clients affiche ID, Nom, IP/Port et État.

Le défi majeur était la mise à jour dynamique depuis les threads réseau de Nolan. Les interfaces Qt ne peuvent être modifiées que depuis le thread principal, or les événements réseau arrivent dans d'autres threads. J'ai résolu ce problème en créant une classe `MasterSignals` utilisant les `pyqtSignal` : le thread réseau émet des signaux (`log_message.emit("...")`), et j'ai connecté ces signaux à mes fonctions d'interface (`signals.log_message.connect(self.add_log)`). Cette approche garantit une communication thread-safe automatiquement gérée par Qt.

J'ai implémenté toutes les fonctions de mise à jour : `add_router()` insère une nouvelle ligne dans le tableau avec les informations du routeur, `add_client()` fait de même pour les clients, `remove_client()` et `remove_router()` suppriment dynamiquement les lignes quand une déconnexion est détectée.

J'ai appliqué un design moderne et cohérent avec la palette de couleurs Catppuccin (fond sombre #1e1e2e, accent bleu #89b4fa, vert pour les statuts actifs #a6e3a1) via les QSS (Qt StyleSheets, similaires au CSS web). Un `QTimer` rafraîchit l'affichage toutes les 2 secondes pour maintenir l'interface à jour.

Interface du Client

Pour le client, j'ai également créé deux fenêtres complètes. La `LoginWindow` gère la connexion avec des champs pour le nom d'utilisateur, l'IP du client, le port d'écoute, l'IP du Master et le port du Master. La validation est particulièrement rigoureuse : vérification du format IP, test de disponibilité du port avec un socket temporaire (pour éviter l'erreur "port déjà utilisé" qui nous bloquait au début), validation des plages numériques, et messages d'erreur explicites via `QMessageBox`.

La `ChatWindow` est la fenêtre principale de messagerie. En haut, une barre d'information affiche le statut de connexion avec les détails du client et du master, un point vert indiquant la connexion active, et un bouton de déconnexion rouge permettant de quitter proprement. Un menu déroulant `QComboBox` permet de sélectionner le destinataire parmi les utilisateurs en ligne récupérés du Master, avec un bouton "refresh" pour actualiser la liste.

La zone centrale affiche les messages sous forme de bulles de conversation modernes. J'ai créé un système d'affichage utilisant HTML/CSS inline dans le QTextEdit : les messages envoyés apparaissent alignés à droite avec un fond vert (#a6e3a1) et le label "Vous • HH:MM", les messages reçus à gauche avec un fond bleu (#89b4fa) et "NomExpéditeur • HH:MM". Chaque bulle a un arrière-plan gris (#45475a), des bordures arrondies (border-radius: 10px) et des marges asymétriques créant l'effet "conversation WhatsApp".

En bas, la barre d'envoi contient trois éléments : un champ numérique pour choisir le nombre de couches de routage (permettant à l'utilisateur de contrôler son niveau d'anonymat, par défaut 1), le champ de texte principal avec placeholder "Tapez votre message...", et un bouton d'envoi stylisé avec une flèche (→) sur fond vert.

Le plus grand défi était l'intégration avec la logique réseau développée par Nolan. J'ai créé une classe `MessageSignals` avec des signaux PyQt spécifiques : `message_received(sender, message, time)` pour les messages entrants et `connection_lost()` pour gérer les déconnexions. J'ai ajouté un attribut `gui_mode` dans la classe `ChatClient` de Nolan pour différencier le comportement en mode CLI (print dans le terminal) et GUI (émission de signaux Qt).

Les threads d'écoute et de keep-alive de Nolan reçoivent maintenant des fonctions callback : quand un message arrive via socket, le thread appelle `callback(sender, message, time)`, qui émet le signal Qt approprié, qui déclenche ma fonction `on_message_received()` dans le thread principal, qui met à jour l'interface de manière sûre. Cette architecture complexe assure une séparation claire entre la logique réseau (Nolan) et l'interface utilisateur (moi).

J'ai implémenté la fonction `display_message()` qui formate chaque message en HTML avec les bonnes couleurs, alignements et timestamps, et utilise `QTextCursor.moveToEnd()` pour scroller automatiquement vers le message le plus récent.

4. Intégration et tests collaboratifs

Une fois que Nolan avait terminé les bases techniques (sockets, chiffrement, routage), nous sommes passés à la phase d'intégration de mes interfaces avec sa logique. J'ai réalisé de nombreux tests pour vérifier que les signaux Qt se déclenchaient correctement lors de la réception de messages via les sockets de Nolan.

Nous avons testé le système complet sur plusieurs machines virtuelles : une pour le Master avec mon interface de monitoring, une pour les routeurs, et plusieurs pour les clients avec mon interface de chat. Cette configuration distribuée validait le déploiement réel de notre système.

Lors des phases de débogage, j'ai identifié et résolu plusieurs problèmes critiques. Par exemple, l'interface se figeait complètement lors de l'envoi de messages : j'ai diagnostiqué que les opérations réseau bloquaient le thread Qt et proposé l'architecture avec signaux que j'ai implémentée. Un autre problème récurrent était les ports déjà utilisés : j'ai ajouté une vérification systématique avec un socket test dans les fenêtres de connexion. Nous avons également résolu ensemble les problèmes d'affichage des caractères spéciaux en ajustant la gestion des codes Unicode hors page.

5. Documentation et livrables

J'ai produit l'ensemble de la documentation utilisateur et technique du projet. Le README principal explique en détail les prérequis (Python 3.8+, MariaDB), la configuration de la base de données (création d'utilisateur, import du schéma), et les commandes de lancement pour chaque composant (Master, Routeurs, Clients).

Le guide utilisateur contient des captures d'écran annotées expliquant comment utiliser l'interface Master (lecture des logs, interprétation des tableaux, arrêt du serveur) et l'interface Client (connexion, sélection du destinataire, choix du nombre de couches, envoi/réception de messages).

J'ai rédigé les sections du rapport final concernant l'architecture des interfaces (détail des classes, widgets utilisés, gestion des signaux), la base de données (justification du schéma, choix techniques), et le bilan complet de gestion de projet (métriques, planning, difficultés rencontrées et solutions).

J'ai également contribué à la vidéo de démonstration en préparant le storyboard (scénario à filmer), en enregistrant les séquences montrant l'utilisation fluide des interfaces Master et Client, et en réalisant le montage avec sous-titres explicatifs pour présenter nos choix techniques.

Analyse de la compétence au regard des AC/CE

AC23.01 | Automatiser l'administration système avec des scripts

Mise en œuvre : J'ai créé les scripts SQL (`schema.sql`) pour automatiser l'initialisation de la base de données, et le script de test Python (`test_db.py`) pour valider la connexion. Dans `master.py`, j'ai implémenté les fonctions `initialize_database()` qui crée automatiquement les tables si elles n'existent pas, et contribué à `clear_database_tables()` qui nettoie les données au démarrage pour garantir un environnement propre. Ces automatisations simplifient le déploiement et la maintenance du système.

Niveau de maîtrise : (3/5) - J'ai acquis de bonnes bases en automatisation SQL/Python. Je pourrais progresser vers des scripts shell plus complexes et l'intégration d'outils de déploiement automatisé.

AC23.02 | Développer une application à partir d'un cahier des charges donné

Mise en œuvre : J'ai analysé précisément le cahier des charges fourni pour identifier les contraintes obligatoires : interfaces graphiques pour Master et Client, visualisation des connexions en temps réel, envoi de messages non bloquant. Mes interfaces répondent exactement à ces spécifications avec un design moderne et une expérience utilisateur soignée. J'ai également respecté les contraintes techniques (PyQt6, pas de code auto-généré, structure claire).

Niveau de maîtrise : (4/5) - Je sais traduire des besoins fonctionnels en interfaces concrètes et ergonomiques. Je peux encore progresser sur l'anticipation des besoins implicites.

AC23.03 | Utiliser un protocole réseau pour programmer une application client/serveur

Mise en œuvre : Bien que Nolan ait développé la logique réseau principale (sockets TCP, threads, protocole de communication), j'ai dû comprendre profondément le protocole pour intégrer correctement mes interfaces. J'ai travaillé sur les callbacks réseau, assuré que les messages reçus via sockets soient affichés correctement dans l'interface, et participé à la définition du format des messages (username::ip::port, PATH:sender:layers:target). J'ai également implémenté la validation réseau côté interface (format IP, disponibilité des ports).

Niveau de maîtrise : (3/5) - J'ai une compréhension fonctionnelle du protocole client-serveur et de l'intégration réseau-interface. La programmation socket bas niveau reste davantage le domaine d'expertise de Nolan.

AC23.04 | Installer, administrer un système de gestion de données

Mise en œuvre : J'ai installé et configuré MariaDB sur ma machine, créé l'utilisateur `routage_user` avec les privilèges appropriés, conçu le schéma relationnel complet (4 tables avec contraintes et index), et mis en place les procédures d'initialisation et nettoyage automatiques. Cette base de données assure la persistance de toutes les informations nécessaires au fonctionnement du système.

Niveau de maîtrise : (4/5) - Je maîtrise la conception de schémas relationnels, l'installation et la configuration. Je pourrais approfondir les aspects avancés comme l'optimisation de performances ou la réplication.

AC23.05 | Accéder à un ensemble de données depuis une application

Mise en œuvre : J'ai implémenté les fonctions d'accès à la base de données dans `master.py` : connexion avec gestion d'erreurs robuste, création automatique des structures, requêtes SELECT pour récupérer routeurs et utilisateurs. J'ai assuré l'intégration entre la BDD et mes interfaces graphiques : les données stockées dans MariaDB sont récupérées et affichées dynamiquement dans les tableaux du Master.

Niveau de maîtrise : (4/5) - Je sais accéder et manipuler des données depuis Python avec gestion d'erreurs appropriée. Je peux progresser sur les requêtes complexes avec jointures et optimisations.

CE3.01 | En étant à l'écoute des besoins du client

Mise en œuvre : En analysant le cahier des charges, j'ai identifié les attentes précises en termes d'expérience utilisateur : interfaces graphiques intuitives, visualisation claire, processus de connexion simple. J'ai priorisé les fonctionnalités essentielles (connexion, envoi/réception, monitoring) et créé des interfaces qui guident l'utilisateur avec validation des entrées, messages d'erreur clairs, et retours visuels immédiats (indicateurs de statut, couleurs, mise à jour en temps réel).

Niveau de maîtrise : (4/5) - J'ai développé une sensibilité à l'expérience utilisateur et une capacité d'analyse des besoins fonctionnels.

CE3.02 | En documentant le travail réalisé

Mise en œuvre : J'ai produit une documentation exhaustive : planning préparatoire détaillé, rapport final de 1800+ lignes, README avec instructions complètes, guide utilisateur illustré, commentaires inline dans tout mon code avec docstrings pour chaque fonction. Mes commit messages suivent les conventions établies, créant un historique Git clair et professionnel.

Niveau de maîtrise : (5/5) - La documentation est un de mes points forts. Mes documents sont complets, structurés, clairs et directement utilisables.

CE3.03 | En utilisant les outils numériques à bon escient

Mise en œuvre : J'ai sélectionné et utilisé efficacement Git/GitHub (versioning, branches, issues pour le suivi), Discord (communication quotidienne, pair programming), VS Code (IDE avec extensions Python, GitLens, MariaDB), Qt Designer (prototypage rapide d'interfaces). Chaque outil avait un usage spécifique et complémentaire, formant un écosystème de développement cohérent.

Niveau de maîtrise : (4/5) - J'utilise efficacement les outils modernes de développement. Je pourrais explorer davantage CI/CD et tests automatisés.

CE3.04 | En choisissant les outils de développement adaptés

Mise en œuvre : J'ai fait des choix technologiques justifiés et complémentaires à ceux de Nolan : PyQt6 pour les interfaces (cross-platform, riche, natif Python), MariaDB pour la persistance (robuste, adapté au multi-utilisateurs), système de signaux Qt pour l'intégration thread-safe. J'ai évalué les alternatives (Tkinter trop basique, SQLite moins adapté) avant de valider ces choix avec Nolan.

Niveau de maîtrise : (4/5) - Mes choix étaient pertinents et bien argumentés. Je peux encore améliorer l'évaluation comparative approfondie des technologies.

CE3.05 | En intégrant les problématiques de sécurité

Mise en œuvre : J'ai intégré plusieurs aspects sécurité : validation systématique des entrées utilisateur (format IP, plage de ports, test de disponibilité), gestion sécurisée des connexions BDD (utilisateur dédié, pas de privilèges root), fermeture propre des ressources pour éviter les fuites. Dans la documentation, j'ai clairement documenté les limitations du RSA simplifié et averti qu'il ne doit PAS être utilisé en production.

Niveau de maîtrise : (3/5) - J'applique les principes de base de la sécurité. Je dois approfondir mes connaissances sur les injections, l'authentification forte et le chiffrement moderne.

Conclusion sur mon apprentissage

Ce projet a été une expérience d'apprentissage extrêmement enrichissante qui m'a permis de développer des compétences à la fois techniques et organisationnelles.

Sur le plan technique, j'ai acquis une compétence solide en développement d'interfaces graphiques avec PyQt6, une technologie totalement nouvelle pour moi il y a six semaines. La progression de

widgets simples jusqu'à la maîtrise de concepts avancés (signaux/slots, threading non-bloquant, communication thread-safe) démontre ma capacité d'apprentissage autonome. J'ai transformé un domaine inconnu en une véritable spécialité, créant des interfaces modernes et professionnelles qui ont suscité des retours très positifs.

J'ai également consolidé mes connaissances en bases de données relationnelles : conception de schémas, contraintes, index, intégration avec l'application. La modélisation pour un système distribué avec gestion d'états dynamiques (online/offline) était particulièrement stimulante.

Sur le plan collaboratif, travailler avec Nolan a été extrêmement formateur. Nos compétences complémentaires (lui sur le réseau/crypto, moi sur les interfaces/données) ont créé une synergie très productive. J'ai appris à communiquer efficacement sur des aspects techniques complexes, à comprendre du code que je n'avais pas écrit, et à intégrer mes développements avec ceux de Nolan. Les sessions de pair programming pour résoudre les problèmes d'intégration interface-réseau m'ont montré la valeur du travail d'équipe et du partage de connaissances.

Sur le plan organisationnel, gérer la structure du projet (Git, documentation, planning) m'a permis de développer des compétences en gestion de projet : anticipation des blocages, ajustement du planning, priorisation des tâches. Le respect de nos délais (livraison le 30 décembre comme prévu) valide l'efficacité de notre organisation.

Mes points forts identifiés : documentation exhaustive et claire, design d'interfaces modernes et ergonomiques, organisation rigoureuse, capacité d'apprentissage rapide, persévérance face aux difficultés techniques, sens de l'expérience utilisateur.

Mes axes d'amélioration : approfondir la programmation réseau et les protocoles, renforcer mes connaissances en cryptographie (au-delà de la compréhension conceptuelle), explorer les tests automatisés et l'intégration continue, et améliorer ma compréhension des problématiques de sécurité avancées.

Ce projet m'a confortée dans mon intérêt pour le développement d'applications avec un accent particulier sur l'expérience utilisateur et le design d'interfaces. La satisfaction de voir nos utilisateurs naviguer intuitivement dans les interfaces que j'ai créées était très gratifiante. Cette expérience m'a également montré l'importance de la polyvalence : être capable de travailler sur différents aspects d'un projet (interfaces, données, organisation) tout en collaborant efficacement avec quelqu'un ayant des compétences complémentaires.

En conclusion, ce projet de 48 jours m'a permis de démontrer ma capacité à concevoir, développer et livrer des interfaces fonctionnelles et professionnelles dans le cadre d'un système distribué complexe. Je suis fière du résultat obtenu, en particulier du feedback positif sur l'ergonomie et l'esthétique de mes interfaces. Les compétences acquises, notamment en PyQt6 et en collaboration technique, me seront très utiles pour la suite de mon parcours académique et professionnel.